

项目二

第一次汇报

数据处理

短期电力负荷预测：数据处理与特征构建

本次只汇报数据处理部分：原始数据理解、清洗重构、缺失值判断、分布分析、异常值检测与处理、2014 年负荷分析、天气合并、特征值演示和参考权重。算法训练和模型选择放到第二次汇报。

1. 原始数据理解

附件 Excel 包含两个核心工作表：**Area_Load** 是电力负荷，**Area_Weather** 是天气。负荷表中每天有 96 个 15 分钟采样点，因此先要把宽表转换成可分析的日粒度数据。

检查项	结果	说明
Area_Load 原始负荷表	2208 行 x 97 列	每天一行，96 个 15 分钟采样点。
Area_Weather 原始天气表	1113 行 x 6 列	每天一行，包含最高/最低/平均温度、湿度、降雨量。
负荷采样列数	96 个	$96 = 24 \text{ 小时} \times \text{每小时 } 4 \text{ 个 } 15 \text{ 分钟点}$ 。
预测目标期负荷缺失	672 个单元格	正好 7 天 x 96 点，是题目要求预测的目标，不作为异常删除。
目标期天气是否完整	0 个缺失	递推预测时可以使用目标期已给定天气。
建模日粒度表	2208 行 x 23 列	由 15 分钟负荷聚合为日最大、日最小、日平均。
IQR 检测字段数	6 个	对负荷目标和主要天气变量做异常值检测。
IQR 极端候选处理方式	保留原值并增加标记	极端天气/负荷有研究价值，不直接删除或截尾。
特征参考权重字段数	66 个	按目标变量分别计算解释性权重。

Area_Load 原始表截图

Area_Load 原始负荷表节选

YMD	T0000	T0015	T0030	T0045	T0100	T0115	T0130	T0145	T0200	T0215	T2330	T2345
20090101	3210.84	3104.51	3037.71	2960.18	2889.23	2827.70	2772.51	2720.47	2666.59	2632.01	2382.70	2276.21
20090102	2163.99	2074.74	1999.62	1950.29	1889.14	1846.49	1800.41	1778.31	1746.28	1716.47	3424.57	3266.95
20090103	3109.13	3015.90	2938.31	2869.38	2815.77	2777.30	2725.38	2704.33	2665.86	2644.08	3602.28	3444.15
20090104	3307.75	3209.73	3125.51	3075.18	3028.71	2995.46	2956.24	2926.63	2893.67	2870.43	3602.40	3477.08
20090105	3295.42	3278.38	3232.31	3160.73	3124.23	3092.29	3064.08	3034.58	3007.34	2984.58	4106.36	3933.51
20090106	3780.43	3678.76	3583.49	3541.04	3473.69	3422.26	3368.35	3343.13	3304.07	3274.83	4111.92	3936.80
20090107	3746.87	3677.80	3608.53	3562.01	3504.42	3453.95	3412.27	3389.51	3350.03	3334.94	4113.67	3955.63
20090108	3786.99	3662.89	3593.44	3542.98	3484.11	3453.87	3414.92	3385.65	3342.56	3323.98	4116.19	3928.63

Area_Weather 原始表截图

Area_Weather 原始天气表节选

YMD	temp_max	temp_min	temp_avg	humidity	rainfall
20120101	19.50	12.10	15.80	63.00	0.00
20120102	20.00	13.00	16.00	59.00	0.00
20120103	18.70	14.20	15.80	72.00	0.00
20120104	14.90	9.30	10.90	62.00	0.00
20120105	9.20	5.10	6.90	78.00	2.90
20120106	11.00	6.30	8.20	92.00	3.30
20120107	12.40	9.40	10.50	80.00	0.00
20120108	16.60	9.90	12.20	75.00	0.00

2. 数据处理主线

处理流程从原始 Excel 出发，先重构负荷表，再聚合为题目要求预测的日最高、日最低、日平均负荷，并与天气表合并。随后不是简单删掉极端值，而是先判断它们是不是“统计极端候选”，再区分物理错误和极端天气样本。

数据处理主线：从 15 分钟负荷宽表到可建模的日粒度特征表



关键原则：只用预测日前已经知道的信息构造滞后/滚动特征；2015-01-11 至 2015-01-17 的负荷缺失是预测目标。

讲解重点：为什么不是直接把 96 个时刻都拿去预测？因为项目目标是预测 2015-01-11 至 2015-01-17 的日最高、日最低、日平均负荷，日粒度聚合能和目标完全对应。

3. 代码截图：读取、重构、聚合、合并

核心代码 1：读取、重构、聚合、合并

```
def load_daily_data() -> pd.DataFrame:
    if not DATA_PATH.exists():
        raise FileNotFoundError(f"Data file not found: {DATA_PATH}")

    # 原始负荷表是“每天一行、96个15分钟采样点”的宽表；建模前先压缩成日粒度。
    # 这样能直接对应题目要求的日最高、日最低、日平均三个预测目标。
    load_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Load")
    weather_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Weather")
    weather_raw.columns = ["YMD", "temp_max", "temp_min", "temp_avg", "humidity", "rainfall"]

    time_cols = [col for col in load_raw.columns if col != "YMD"]
    long_df = load_raw.melt(id_vars="YMD", value_vars=time_cols, var_name="time_slot", value_name="load")
    daily = (
        long_df.groupby("YMD", as_index=False)
        .agg(load_max=("load", "max"), load_min=("load", "min"), load_mean=("load", "mean"))
        .merge(weather_raw, on="YMD", how="left")
    )
    daily["date"] = pd.to_datetime(daily["YMD"].astype(str), format="%Y%m%d")
    return add_features(daily.sort_values("date").reset_index(drop=True))
```

推导解释

原始负荷宽表的每一行是一日，列 T0000 到 T2345 表示一天内 96 个采样点。使用 `melt` 将宽表变成长表后，每条记录对应“某一天某一时刻的负荷”；再按 YMD 分组，分别取最大值、最小值和平均值，得到三个预测目标。

```
def load_daily_data() -> pd.DataFrame:
    if not DATA_PATH.exists():
        raise FileNotFoundError(f"Data file not found: {DATA_PATH}")
```

```

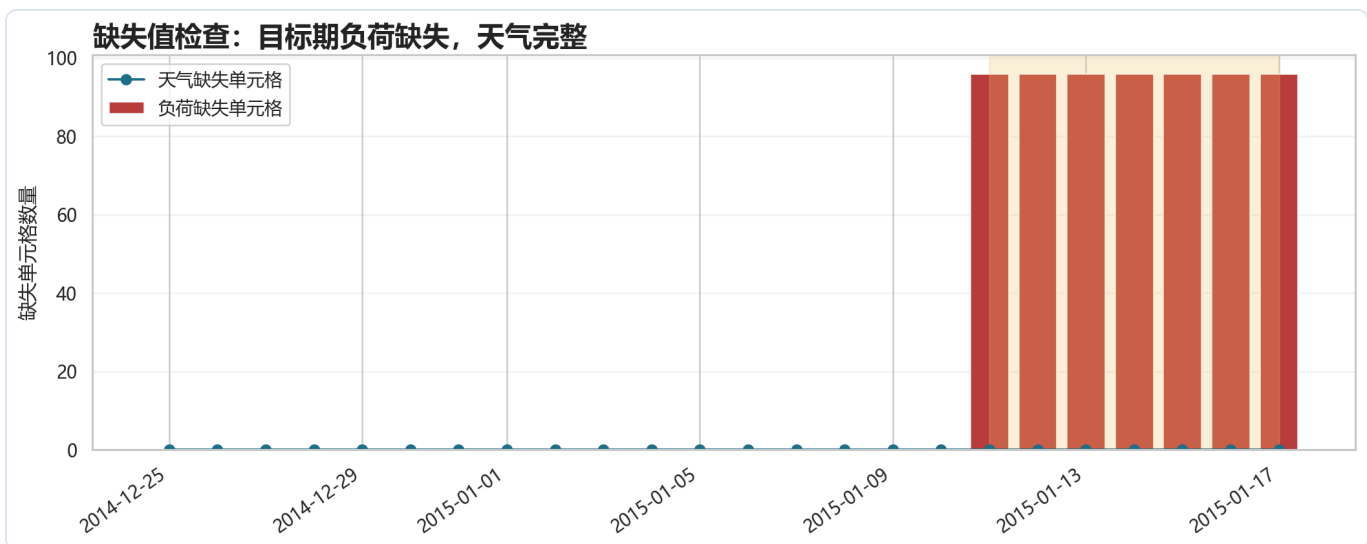
# 原始负荷表是“每天一行、96个15分钟采样点”的宽表；建模前先压缩成日粒度。
# 这样能直接对应题目要求的日最高、日最低、日平均三个预测目标。
load_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Load")
weather_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Weather")
weather_raw.columns = ["YMD", "temp_max", "temp_min", "temp_avg", "humidity",
"rainfall"]

time_cols = [col for col in load_raw.columns if col != "YMD"]
long_df = load_raw.melt(id_vars="YMD", value_vars=time_cols, var_name="time_slot",
value_name="load")
daily = (
    long_df.groupby("YMD", as_index=False)
    .agg(load_max=("load", "max"), load_min=("load", "min"), load_mean=("load",
"mean"))
    .merge(weather_raw, on="YMD", how="left")
)
daily["date"] = pd.to_datetime(daily["YMD"].astype(str), format="%Y%m%d")
return add_features(daily.sort_values("date").reset_index(drop=True))

```

4. 缺失值检查与业务判断

2015-01-11 至 2015-01-17 的负荷缺失不是脏数据，而是题目要求预测的未来目标；天气数据在这一段完整，可以作为预测输入。

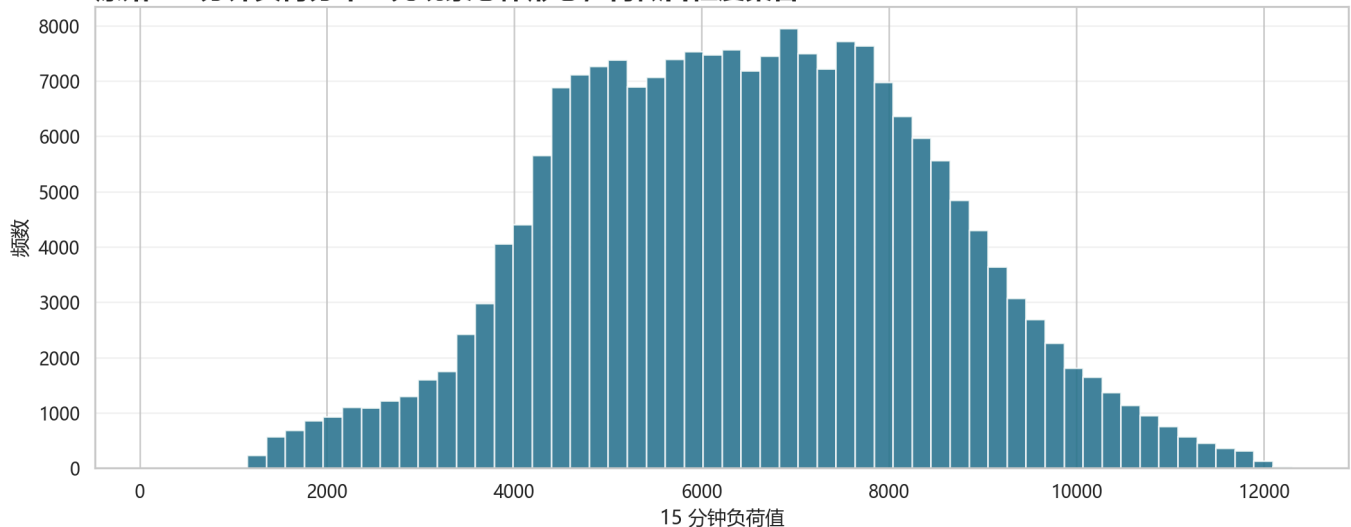


结论：不删除目标期 7 天，也不使用插值填补目标负荷；后续预测要用递推方式生成这 7 天的负荷。

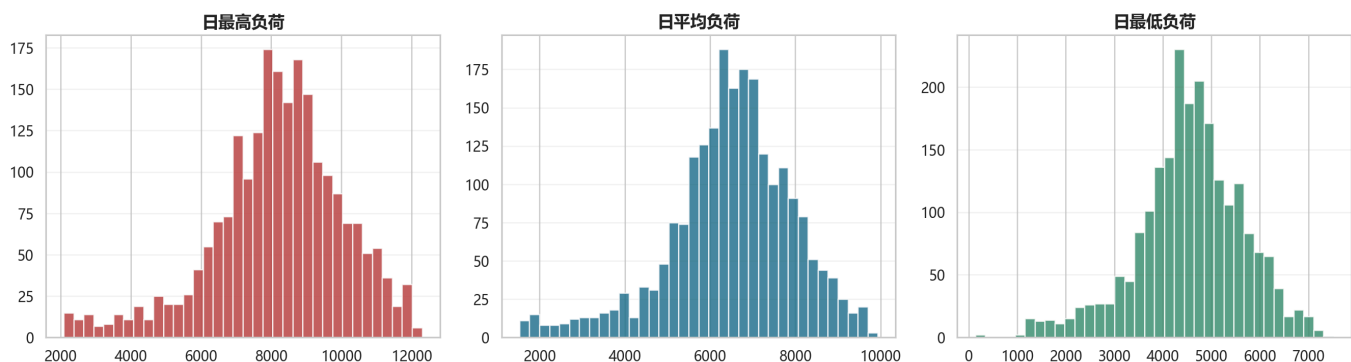
5. 数据分布分析

先看原始 15 分钟负荷的总体分布，再看聚合后的日最高、日平均、日最低负荷分布，最后检查天气变量分布。分布图的作用是判断变量是否偏态、是否存在长尾，以及后续异常值处理是否必要。

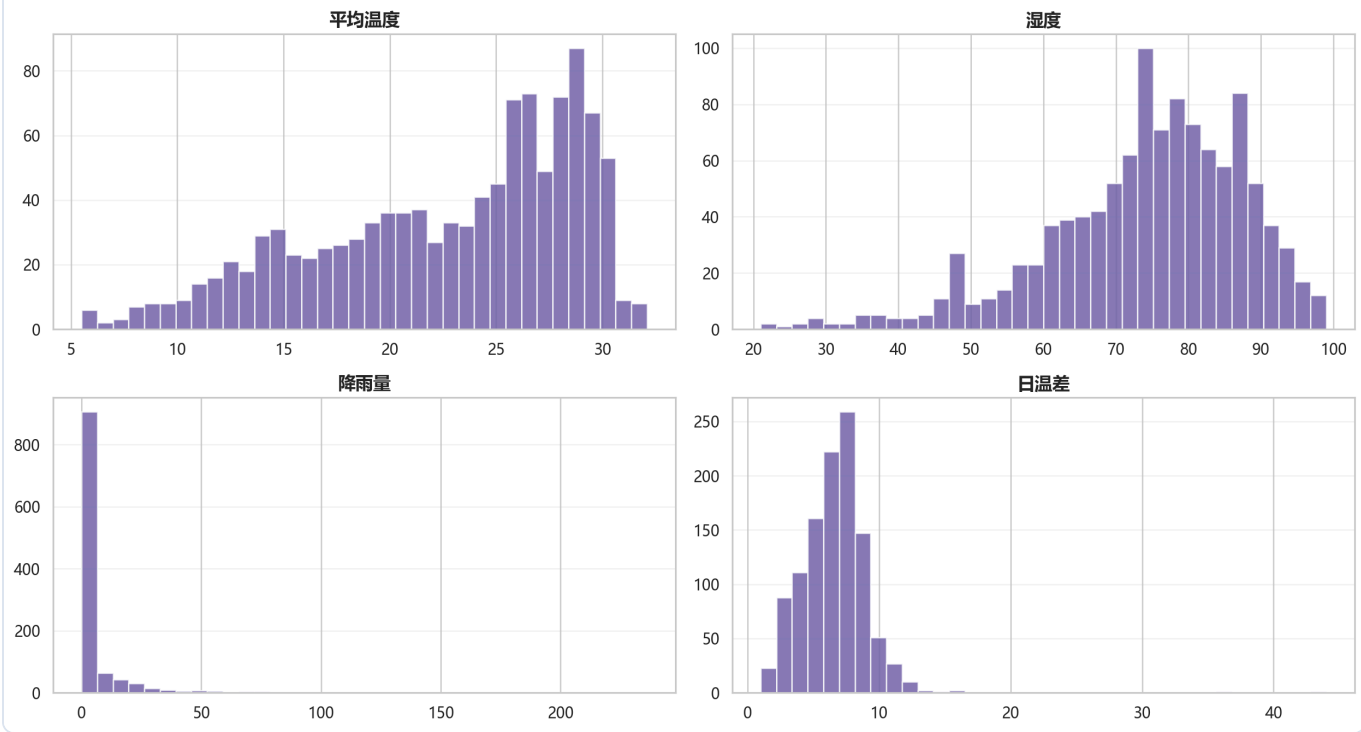
原始 15 分钟负荷分布：先观察总体形态，再做日粒度聚合



日粒度负荷分布：三个预测目标的取值范围和偏态



天气变量分布：检查气象输入的范围和偏态



6. 极端值识别与质量检查

IQR 四分位距规则适合发现“统计极端候选值”：下界 = $Q1 - 1.5 \times IQR$ ，上界 = $Q3 + 1.5 \times IQR$ 。但在电力负荷预测中，极端天气和极端负荷不一定是错误，反而往往最有研究价值。因此本项目不把 IQR 候选值直接删除或截尾，而是保留原始值，并增加极端事件标记。

物理不可能值检查

真正需要修正的是物理不可能或逻辑不成立的数据，例如负负荷、湿度超过 100、负降雨、最低温高于平均温等。

检查对象	规则	问题数量	处理结论
15 分钟负荷	负荷应大于 0	0	若出现才视为错误；当前不需要修正
湿度	$0 \leq \text{humidity} \leq 100$	0	越界才视为错误；当前不需要修正
降雨量	$\text{rainfall} \geq 0$	0	负降雨才视为错误；当前不需要修正

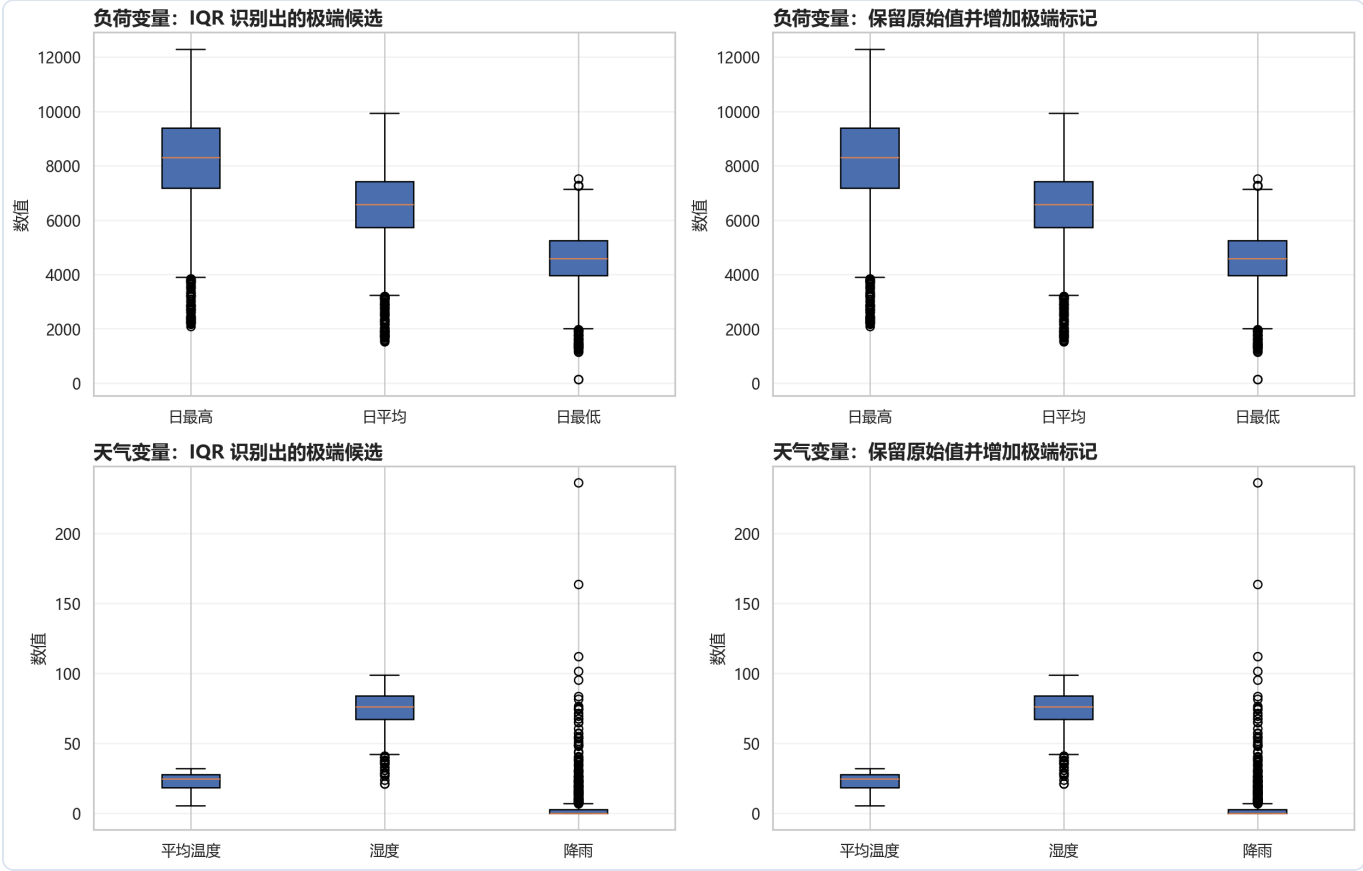
检查对象	规则	问题数量	处理结论
温度关系	temp_min <= temp_avg <= temp_max	0	关系不成立才视为错误；当前不需要修正

IQR 极端候选值统计

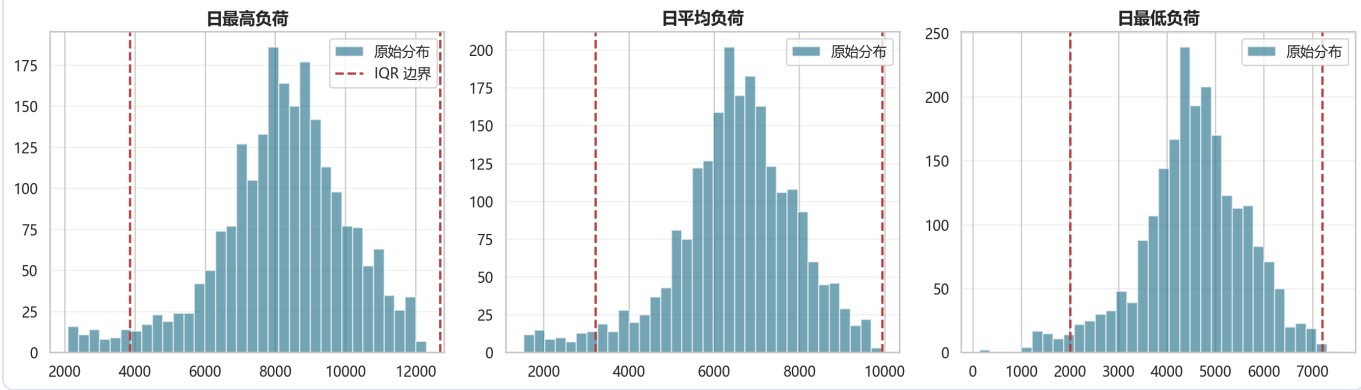
字段	中文含义	Q1	Q3	IQR	下界	上界	极端候选数量	极端占比 (%)	业务解释	处理策略
load_max	日最高负荷	7172.22	9383.88	2211.66	3854.72	12701.37	72	3.27	可能是极端天气、节假日或生产活动变化导致	保留原始值，增加极端事件标记，不作为错误删除
load_min	日最低负荷	3953.46	5253.80	1300.34	2002.95	7204.31	61	2.77	可能是极端天气、节假日或生产活动变化导致	保留原始值，增加极端事件标记，不作为错误删除
load_mean	日平	5734.89	7417.45	1682.56	3211.05	9941.30	79	3.59	可能是极	保留原始

字段	中文含义	Q1	Q3	IQR	下界	上界	极端候选数量	极端占比(%)	业务解释	处理策略
	均负荷								端天气、节假日或生产活动变化导致	值, 增加极端事件标记, 不作为错误删除
temp_avg	平均温度	18.20	27.90	9.70	3.65	42.45	0	0.00	可能是极端天气、节假日或生产活动变化导致	保留原始值, 增加极端事件标记, 不作为错误删除
humidity	相对湿度	67.00	84.00	17.00	41.50	109.50	28	2.53	可能是极端天气、节假日或生产活动变化导致	保留原始值, 增加极端事件标记, 不作为错误删除

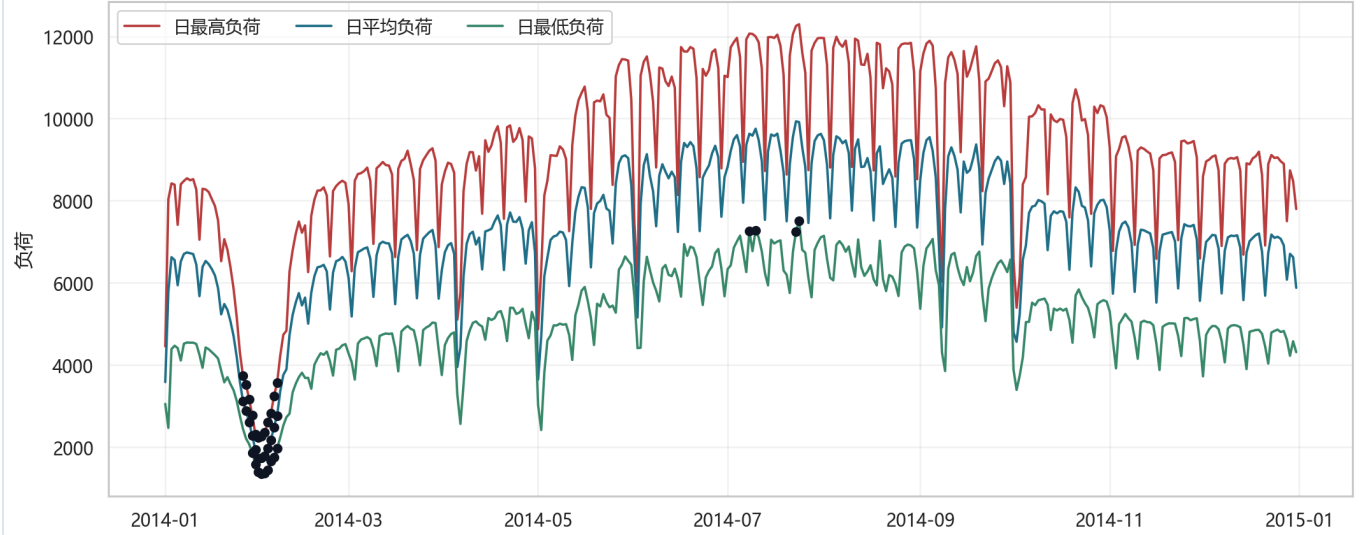
字段	中文含义	Q1	Q3	IQR	下界	上界	极端候选数量	极端占比 (%)	业务解释	处理策略
rainfall	降雨量	0.00	2.80	2.80	-4.20	7.00	195	17.66	可能是极端天气、节假日或生产活动变化导致	保留原始值，增加极端事件标记，不作为错误删除



负荷分布与 IQR 边界：极端值保留，边界仅用于标记



2014 年极端事件位置：黑点为 IQR 规则识别出的候选日期



代码截图：极端值识别与质量检查

核心代码 2：IQR 异常值检测

```
def detect_outliers_iqr(df: pd.DataFrame, columns: list[str]) -> pd.DataFrame:
    """使用 IQR 四分位距规则识别“极端候选值”，对应 HTML 第 6 节。
```

注意：IQR 标出来的是统计意义上的极端值，不等于错误数据。

对电力负荷预测来说，极端天气和极端负荷反而很有研究价值，所以这里的策略是“识别并标记”，不是直接删除或截尾。

```
"""
```

只用已知历史期计算异常边界，不能把 2015-01-11 之后的预测目标期混进去。

```
known = df[df["date"] <= pd.Timestamp("2015-01-10")].copy()
```

```
rows: list[dict[str, object]] = []
```

```
for col in columns:
```

```
    values = known[col].dropna()
```

```
    # Q1/Q3 是 25% 和 75% 分位数; IQR = Q3 - Q1。
```

```
    q1 = float(values.quantile(0.25))
```

```
    q3 = float(values.quantile(0.75))
```

```
    iqr = q3 - q1
```

```
    # 经典箱线图异常值边界：低于 Q1-1.5IQR 或高于 Q3+1.5IQR。
```

```
    lower = q1 - 1.5 * iqr
```

```
    upper = q3 + 1.5 * iqr
```

```
    flags = (values < lower) | (values > upper)
```

```
    rows.append(
```

```
        {
```

```
            "字段": col,
```

```
            "中文含义": feature_label(col),
```

```
            "Q1": q1,
```

```
            "Q3": q3,
```

```
            "IQR": iqr,
```

```
            "下界": lower,
```

```
            "上界": upper,
```

```
            "极端候选数量": int(flags.sum()),
```

```
            "极端占比(%)": round(float(flags.mean() * 100), 2),
```

```
            "业务解释": "可能是极端天气、节假日或生产活动变化导致",
```

```
            "处理策略": "保留原始值，增加极端事件标记，不作为错误删除",
```

```
        }
    )
return pd.DataFrame(rows)
```

```
def detect_outliers_iqr(df: pd.DataFrame, columns: list[str]) -> pd.DataFrame:
```

```
    """使用 IQR 四分位距规则识别“极端候选值”，对应 HTML 第 6 节。
```

注意：IQR 标出来的是统计意义上的极端值，不等于错误数据。

对电力负荷预测来说，极端天气和极端负荷反而很有研究价值，

所以这里的策略是“识别并标记”，不是直接删除或截尾。

```
"""
```

只用已知历史期计算异常边界，不能把 2015-01-11 之后的预测目标期混进去。

```
known = df[df["date"] <= pd.Timestamp("2015-01-10")].copy()
```

```
rows: list[dict[str, object]] = []
```

```
for col in columns:
```

```
    values = known[col].dropna()
```

```
    # Q1/Q3 是 25% 和 75% 分位数; IQR = Q3 - Q1。
```

```
    q1 = float(values.quantile(0.25))
```

```
    q3 = float(values.quantile(0.75))
```

```
    iqr = q3 - q1
```

```
    # 经典箱线图异常值边界：低于 Q1-1.5IQR 或高于 Q3+1.5IQR。
```

```
    lower = q1 - 1.5 * iqr
```

```
    upper = q3 + 1.5 * iqr
```

```
    flags = (values < lower) | (values > upper)
```

```
    rows.append(
```

```

        {
            "字段": col,
            "中文含义": feature_label(col),
            "Q1": q1,
            "Q3": q3,
            "IQR": iqr,
            "下界": lower,
            "上界": upper,
            "极端候选数量": int(flags.sum()),
            "极端占比(%)": round(float(flags.mean() * 100), 2),
            "业务解释": "可能是极端天气、节假日或生产活动变化导致",
            "处理策略": "保留原始值，增加极端事件标记，不作为错误删除",
        }
    )
    return pd.DataFrame(rows)

def add_extreme_event_flags(df: pd.DataFrame, outlier_summary: pd.DataFrame) ->
pd.DataFrame:
    """根据 IQR 边界增加极端事件标记，但不改变原始数值。"""
    cleaned = df.copy()
    for _, row in outlier_summary.iterrows():
        col = str(row["字段"])
        lower = float(row["下界"])
        upper = float(row["上界"])
        # 标记字段用于解释和后续建模扩展：原始负荷/天气值全部保留。
        cleaned[f"{col}_extreme_flag"] = ((cleaned[col] < lower) | (cleaned[col] >
upper)).astype(int)
    return cleaned

def physical_quality_checks(load_raw: pd.DataFrame, daily: pd.DataFrame) ->
pd.DataFrame:
    """检查真正需要修正的物理不可能值，例如负负荷、湿度越界。"""
    time_cols = [c for c in load_raw.columns if c != "YMD"]
    known_load = load_raw[load_raw["YMD"] <= 20150110][time_cols]
    rows = [
        {
            "检查对象": "15 分钟负荷",
            "规则": "负荷应大于 0",
            "问题数量": int((known_load <= 0).sum().sum()),
            "处理结论": "若出现才视为错误；当前不需要修正",
        },
        {
            "检查对象": "湿度",

```

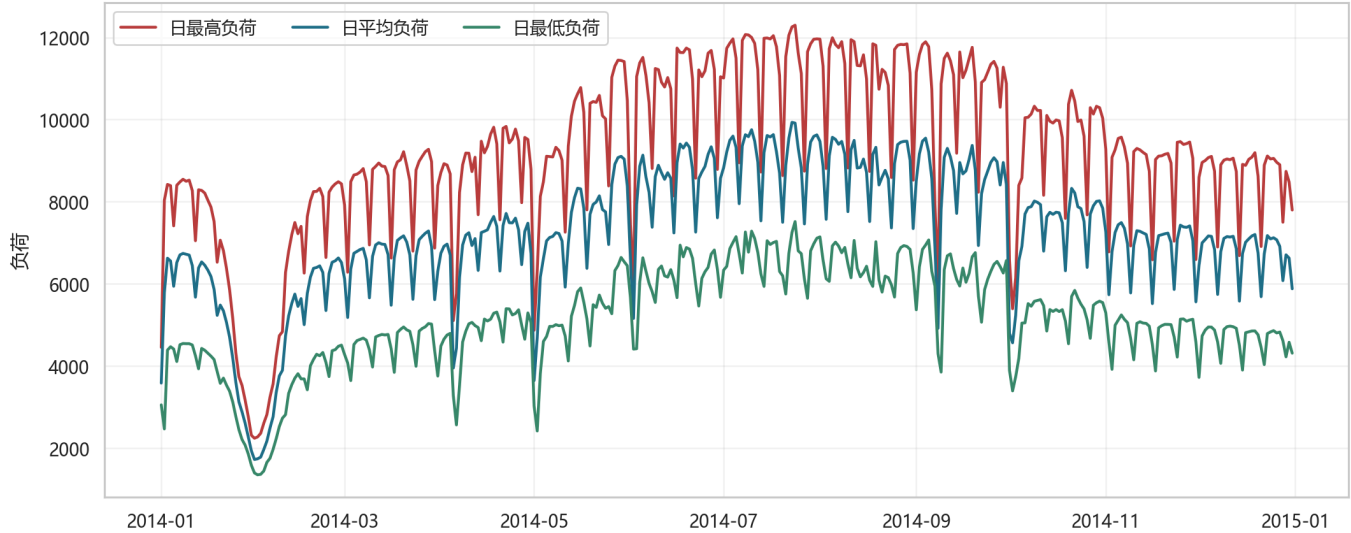
```
"规则": "0 <= humidity <= 100",
"问题数量": int(((daily["humidity"] < 0) | (daily["humidity"] > 100)).sum()),
"处理结论": "越界才视为错误：当前不需要修正",
},
{
    "检查对象": "降雨量",
    "规则": "rainfall >= 0",
    "问题数量": int((daily["rainfall"] < 0).sum()),
    "处理结论": "负降雨才视为错误：当前不需要修正",
},
{
    "检查对象": "温度关系",
    "规则": "temp_min <= temp_avg <= temp_max",
    "问题数量": int(((daily["temp_min"] > daily["temp_avg"]) | (daily["temp_avg"] >
daily["temp_max"]))).sum()),
    "处理结论": "关系不成立才视为错误：当前不需要修正",
},
]
return pd.DataFrame(rows)
```

汇报口径：IQR 只是提醒我们“这里值得解释”，不是判定错误的最终标准。极端天气对应的负荷变化是预测模型必须学习的场景，所以应保留并标记。

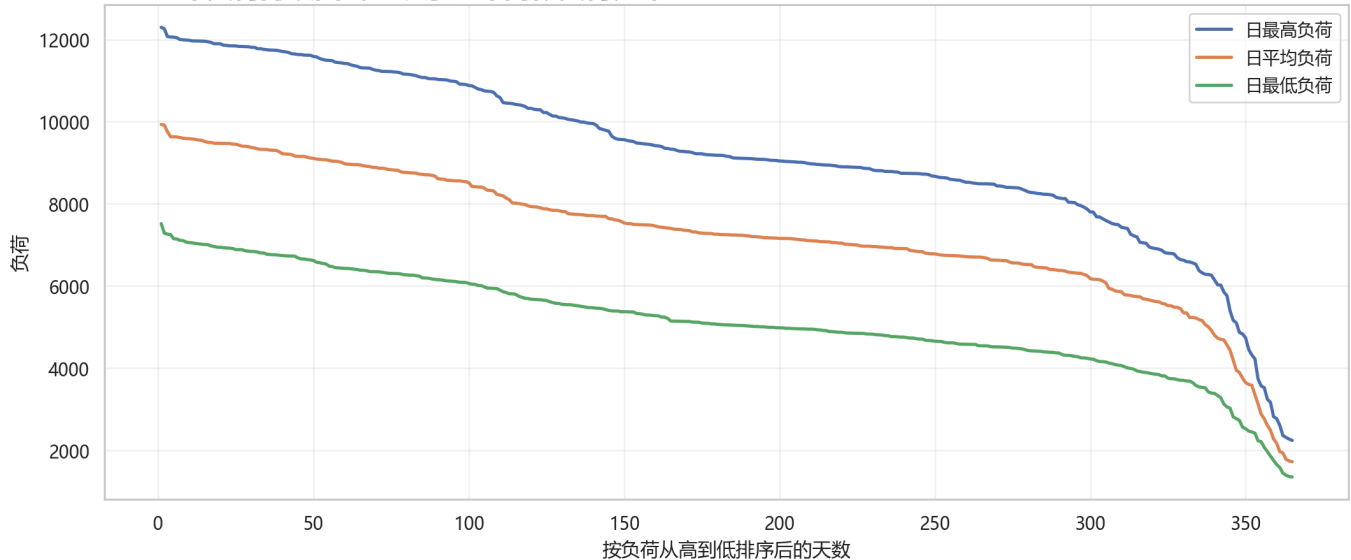
7. 2014 年负荷分析

按照实训要求，对 2014 年的日负荷进行分析。日最高、日平均、日最低三条曲线可以展示季节性和短期波动；负荷持续曲线可以展示全年高负荷和低负荷分布。

2014 年日负荷曲线：保留季节性、周周期和极值变化



2014 年负荷持续曲线：观察全年高低负荷分布



8. 天气合并与衍生变量

这一部分不是凭空加特征，而是从业务机理和数据观察共同推出来的。电力负荷会受到温度影响：高温时空调制冷增加，低温时供热或取暖设备增加，湿度和降雨也会改变居民、商业和工业用电行为。因此原始天气表不能孤立放着，需要按日期 YMD 合并到日负荷表中。

怎么想到的：业务联想

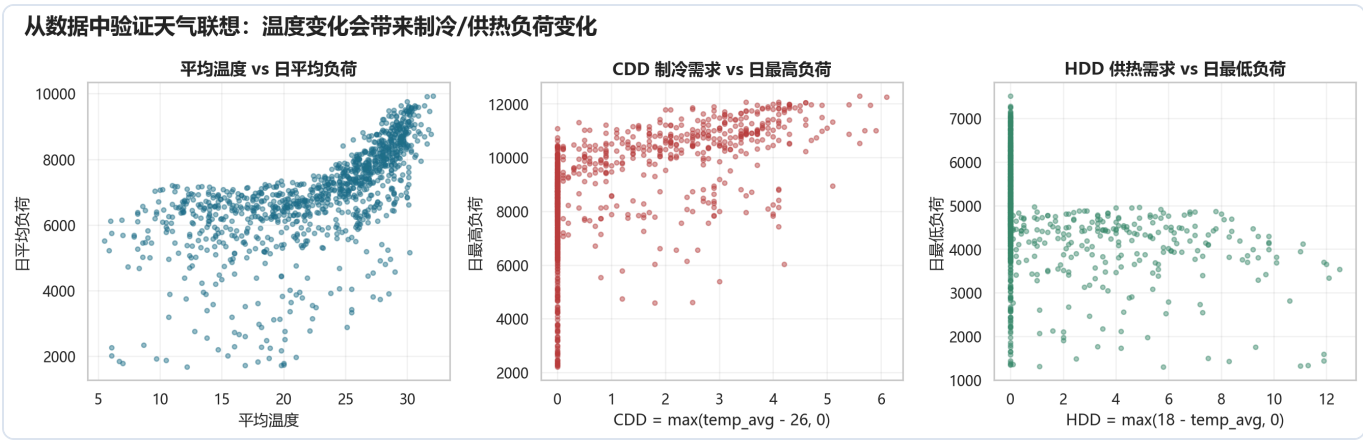
负荷预测的目标是“某一天”的最高、最低、平均负荷，而天气表也是“某一天”的天气记录，两者天然可以用日期对齐。进一步看，单独的平均温度还不够表达用电机理，所以构造三个衍生

变量：

```
temp_range = temp_max - temp_min：表示一天内温差，温差大时负荷波动可能更明显。  
hdd = max(18 - temp_avg, 0)：低于舒适温度时的供热需求强度。  
cdd = max(temp_avg - 26, 0)：高于高温阈值时的制冷需求强度。
```

怎么发现的：数据验证

先用散点图验证天气和负荷是否存在关系。图中可以看到，温度、CDD、HDD 与不同负荷目标存在可观察的关联，因此把天气变量加入特征表是有依据的。



怎么实现的：日期合并 + 衍生变量

实现上分两步：第一步在 load_daily_data() 中把负荷宽表聚合成日粒度，再按 YMD 和天气表 merge；第二步在 add_features() 中计算 temp_range、hdd、cdd。

日粒度负荷 + 天气 + 衍生变量节选

YMD	load_max	load_min	load_mean	temp_max	temp_min	temp_avg	humidity	rainfall	temp_range	hdd	cdd
20090101	3210.84	2122.43	2573.97								
20090102	4865.44	1616.35	3479.02								
20090103	5183.23	2489.03	4026.50								
20090104	5164.95	2679.78	4076.88								
20090105	5852.40	2822.75	4548.92								
20090106	5909.08	3082.16	4680.46								
20090107	5816.91	3117.07	4675.43								
20090108	5867.26	3133.23	4668.18								
20090109	5893.13	3156.11	4697.20								
20090110	5550.50	3092.10	4400.61								

代码体现 1：按 YMD 合并天气

核心代码 1：读取、重构、聚合、合并

```
def load_daily_data() -> pd.DataFrame:
    if not DATA_PATH.exists():
        raise FileNotFoundError(f"Data file not found: {DATA_PATH}")

    # 原始负荷表是“每天一行、96个15分钟采样点”的宽表；建模前先压缩成日粒度。
    # 这样能直接对应题目要求的日最高、日最低、日平均三个预测目标。
    load_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Load")
    weather_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Weather")
    weather_raw.columns = ["YMD", "temp_max", "temp_min", "temp_avg", "humidity", "rainfall"]

    time_cols = [col for col in load_raw.columns if col != "YMD"]
    long_df = load_raw.melt(id_vars="YMD", value_vars=time_cols, var_name="time_slot", value_name="load")
    daily = (
        long_df.groupby("YMD", as_index=False)
        .agg(load_max=("load", "max"), load_min=("load", "min"), load_mean=("load", "mean"))
        .merge(weather_raw, on="YMD", how="left")
    )
    daily["date"] = pd.to_datetime(daily["YMD"].astype(str), format="%Y%m%d")
    return add_features(daily.sort_values("date").reset_index(drop=True))
```

代码体现 2：构造天气衍生变量**核心代码 3：日历周期与天气衍生特征**

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["dayofweek"] = df["date"].dt.dayofweek
    df["month"] = df["date"].dt.month
    df["day_of_year"] = df["date"].dt.dayofyear
    df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)

    # 周期特征不能直接用“月份=12、1”这种数值距离，因为12月和1月在时间上相邻。
    # sin/cos 编码把周期映射到圆上，保留“首尾相接”的季节/星期规律。
    df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
    df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
    df["dow_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
    df["dow_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)
    df["doy_sin"] = np.sin(2 * np.pi * df["day_of_year"] / 365)
    df["doy_cos"] = np.cos(2 * np.pi * df["day_of_year"] / 365)

    # HDD/CDD 是电力负荷常用气象衍生变量：低温带来供热需求，高温带来制冷需求。
    df["temp_range"] = df["temp_max"] - df["temp_min"]
    df["hdd"] = np.maximum(18 - df["temp_avg"], 0)
    df["cdd"] = np.maximum(df["temp_avg"] - 26, 0)
    return df
```

9. 代码截图：日历、周期和天气特征

核心代码 3：日历周期与天气衍生特征

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["dayofweek"] = df["date"].dt.dayofweek
    df["month"] = df["date"].dt.month
    df["day_of_year"] = df["date"].dt.dayofyear
    df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)

    # 周期特征不能直接用“月份=12、1”这种数值距离，因为12月和1月在时间上相邻。
    # sin/cos 编码把周期映射到圆上，保留“首尾相接”的季节/星期规律。
    df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
    df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
    df["dow_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
    df["dow_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)
    df["doy_sin"] = np.sin(2 * np.pi * df["day_of_year"] / 365)
    df["doy_cos"] = np.cos(2 * np.pi * df["day_of_year"] / 365)

    # HDD/CDD 是电力负荷常用气象衍生变量：低温带来供热需求，高温带来制冷需求。
    df["temp_range"] = df["temp_max"] - df["temp_min"]
    df["hdd"] = np.maximum(18 - df["temp_avg"], 0)
    df["cdd"] = np.maximum(df["temp_avg"] - 26, 0)
    return df
```

推导解释

月份、星期和年内日序号都是周期变量。如果直接把 12 月和 1 月当普通数字，模型会误以为二者距离很远；所以使用 $\sin/\cos$ 编码，把周期映射到圆上，保留首尾相接的季节规律。

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["dayofweek"] = df["date"].dt.dayofweek
    df["month"] = df["date"].dt.month
    df["day_of_year"] = df["date"].dt.dayofyear
    df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)

    # 周期特征不能直接用“月份=12、1”这种数值距离，因为12月和1月在时间上相邻。
    # sin/cos 编码把周期映射到圆上，保留“首尾相接”的季节/星期规律。
    df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
    df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
    df["dow_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
    df["dow_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)
    df["doy_sin"] = np.sin(2 * np.pi * df["day_of_year"] / 365)
    df["doy_cos"] = np.cos(2 * np.pi * df["day_of_year"] / 365)

    # HDD/CDD 是电力负荷常用气象衍生变量：低温带来供热需求，高温带来制冷需求。
    df["temp_range"] = df["temp_max"] - df["temp_min"]
    df["hdd"] = np.maximum(18 - df["temp_avg"], 0)
    df["cdd"] = np.maximum(df["temp_avg"] - 26, 0)
    return df
```

10. 代码截图：滞后与滚动特征

特征工程结构：让模型同时看到天气、日历和历史负荷惯性



说明：滚动统计使用 shift(1) 后再 rolling，表示今天预测时只使用昨天及以前的真实历史，避免数据泄漏。

核心代码 4：滞后与滚动特征，避免数据泄漏

```
def make_target_features(df: pd.DataFrame, target: str) -> tuple[pd.DataFrame, list[str]]:
    out = df.copy()
    lag_features: list[str] = []

    # 负荷序列有强惯性和周周期：昨天、上周同日、两周前同日通常是强预测信号。
    # 每个目标单独构造滞后项，避免用“日均负荷”的历史去预测“日最高/最低”时混淆目标。
    for lag in [1, 7, 14]:
        col = f'{target}_lag_{lag}'
        out[col] = out[target].shift(lag)
        lag_features.append(col)

    # 滚动均值/标准差描述近期负荷水平和波动程度，是短期预测里很实用的平滑特征。
    for win in [7, 14]:
        mean_col = f'{target}_roll_mean_{win}'
        std_col = f'{target}_roll_std_{win}'
        out[mean_col] = out[target].shift(1).rolling(win).mean()
        out[std_col] = out[target].shift(1).rolling(win).std()
        lag_features.extend([mean_col, std_col])
    return out, MODEL_WEATHER + CALENDAR_FEATURES + lag_features
```

推导解释

电力负荷有明显惯性：昨天、上周同日、两周前同日通常是强信号。滚动均值和标准差用于描述近期水平和波动。所有滚动统计都先 shift(1)，保证今天的预测不会偷看今天真实负荷，避免数据泄漏。

```
def make_target_features(df: pd.DataFrame, target: str) -> tuple[pd.DataFrame, list[str]]:
    out = df.copy()
    lag_features: list[str] = []

    # 负荷序列有强惯性和周周期：昨天、上周同日、两周前同日通常是强预测信号。
    # 每个目标单独构造滞后项，避免用“日均负荷”的历史去预测“日最高/最低”时混淆目标。
    for lag in [1, 7, 14]:
```

```
col = f"{target}_lag_{lag}"
out[col] = out[target].shift(lag)
lag_features.append(col)

# 滚动均值/标准差描述近期负荷水平和波动程度，是短期预测里很实用的平滑特征。
for win in [7, 14]:
    mean_col = f"{target}_roll_mean_{win}"
    std_col = f"{target}_roll_std_{win}"
    out[mean_col] = out[target].shift(1).rolling(win).mean()
    out[std_col] = out[target].shift(1).rolling(win).std()
    lag_features.extend([mean_col, std_col])
return out, MODEL_WEATHER + CALENDAR_FEATURES + lag_features
```

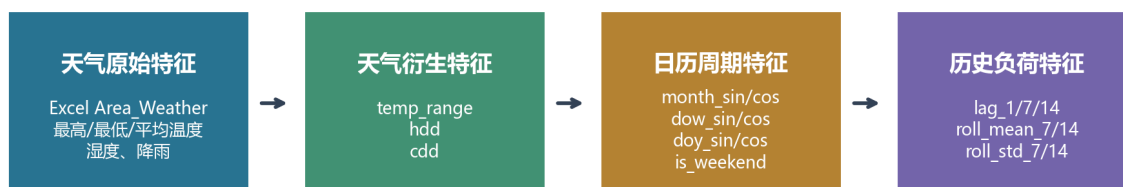
11. 特征值演示与参考权重

下表展示每个建模特征的中文含义、示例日期、示例值 and 数据处理阶段的参考权重。这里的特征不是手工随便写出来的，而是由代码中的特征列表自动生成：MODEL_WEATHER + CALENDAR_FEATURES + lag_features。

特征名是哪来的

特征来源分为四类：第一类是天气表原始字段，如温度、湿度、降雨；第二类是基于业务机理构造的天气衍生变量，如 temp_range、hdd、cdd；第三类是日历周期变量，用来表达月份、星期和季节周期；第四类是历史负荷变量，用来表达昨天、上周同日、两周前同日以及近期滚动水平。

特征名来源：从原始数据、业务衍生、日历周期和历史负荷自动组合



代码中最终特征列表 = MODEL_WEATHER + CALENDAR_FEATURES + lag_features；特征演示表从这份列表自动生成，不是手工编出来的。

数学含义怎么和现实结合

数学上，`lag_7` 是前 7 天的目标值；现实上，它代表“上周同一天的用电习惯”。

`roll_mean_7` 是过去 7 天均值；现实上，它代表近期用电水平。

`roll_std_7` 是过去 7 天标准差；现实上，它代表近期负荷波动程度。

`month_sin/cos`、`dow_sin/cos` 是周期编码；现实上，它们代表季节和星期规律。

和天气衍生变量的关系

第 8 节讲的是天气变量为什么要合并、为什么要衍生；第 11 节是在此基础上把所有可建模特征统一列出来。也就是说，`temp_range`、`hdd`、`cdd` 是特征体系中的“天气衍生特征”部分，后面再和日历特征、历史负荷特征一起进入建模数据表。

示例值怎么获取

示例值来自历史期最后一条完整样本，即 2015-01-10。选择这一天是因为它紧挨着预测窗口 2015-01-11，最适合说明“预测第一天时，模型能看到哪些已知信息”。

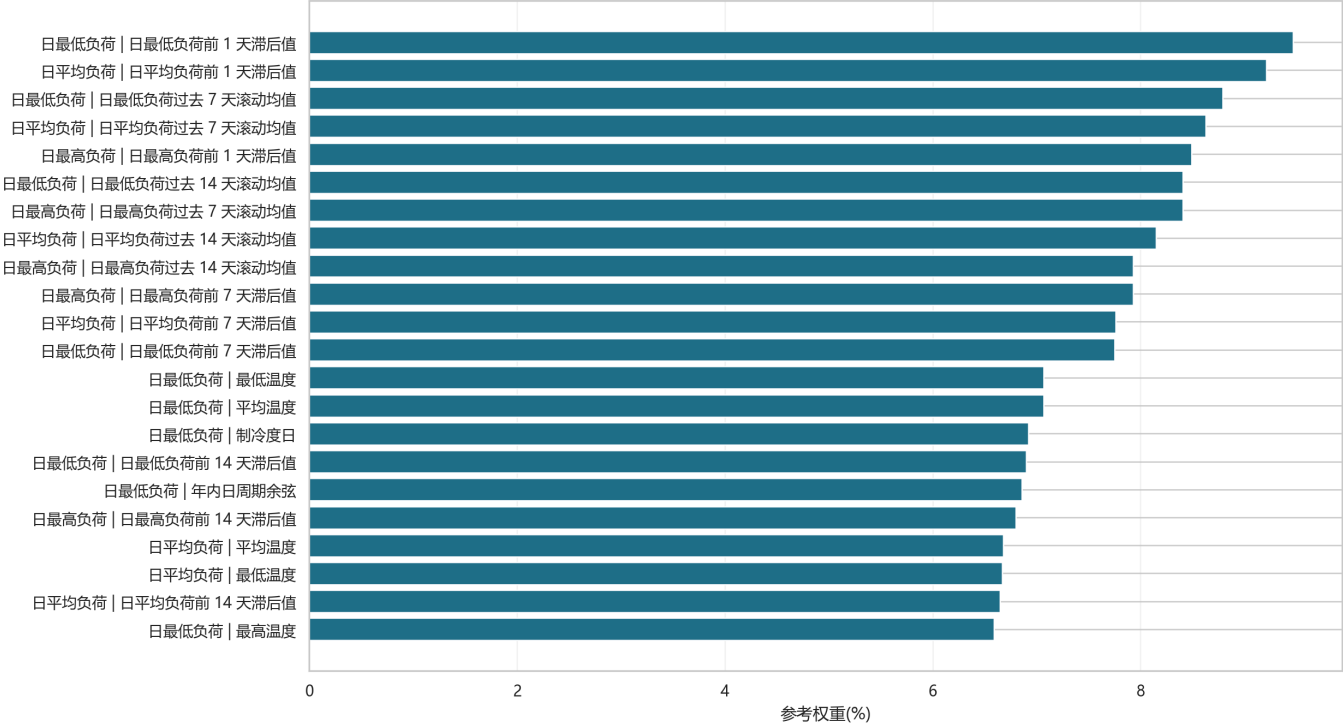
参考权重怎么计算

权重不是最终模型参数，而是用每个特征和预测目标之间的单变量 Pearson 相关系数计算：先取相关系数绝对值表示关系强弱，再在同一目标变量下归一化成百分比。因此它只用于数据处理阶段的解释性排序。

特征值演示节选

目标变量	特征名	中文含义	示例日期	示例值	参考权重(%)	相关方向
日平均负荷	load_mean_lag_1	日平均负荷前 1 天滞后值	2015-01-10	7086.06	9.21	正相关
日平均负荷	load_mean_rolI_mean_7	日平均负荷过去 7 天滚动均值	2015-01-10	6693.03	8.63	正相关
日平均负荷	load_mean_rolI_mean_14	日平均负荷过去 14 天滚动均值	2015-01-10	6235.96	8.15	正相关
日平均负荷	load_mean_lag_7	日平均负荷前 7 天滞后值	2015-01-10	5934.02	7.76	正相关
日平均负荷	temp_avg	平均温度	2015-01-10	15.40	6.68	正相关
日平均负荷	temp_min	最低温度	2015-01-10	12.00	6.67	正相关
日平均负荷	load_mean_lag_14	日平均负荷前 14 天滞后值	2015-01-10	6912.88	6.65	正相关
日平均负荷	doy_cos	年内日周期余弦	2015-01-10	0.99	6.51	负相关
日平均负荷	cdd	制冷度日	2015-01-10	0.00	6.40	正相关
日平均负荷	temp_max	最高温度	2015-01-10	19.90	6.23	正相关
日平均负荷	month_cos	月份周期余弦	2015-01-10	0.87	5.33	负相关
日平均负荷	month_sin	月份周期正弦	2015-01-10	0.50	5.11	负相关
日平均负荷	hdd	供热量日	2015-01-10	2.60	3.90	负相关
日平均负荷	doy_sin	年内日周期正弦	2015-01-10	0.17	3.58	负相关
日平均负荷	is_weekend	是否周末	2015-01-10	1.00	2.12	负相关
日平均负荷	dow_sin	星期周期正弦	2015-01-10	-0.97	1.62	正相关
日平均负荷	dow_cos	星期周期余弦	2015-01-10	-0.22	1.41	负相关
日平均负荷	humidity	相对湿度	2015-01-10	54.00	1.23	正相关

特征参考权重 Top 22：基于单变量相关性的解释性排序



目标变量	特征名	中文含义	示例日期	示例值	参考权重(%)	相关方向
日平均负荷	load_mean_lag_1	日平均负荷前 1 天滞后值	2015-01-10	7086.06	9.21	正相关
日平均负荷	load_mean_roll_mean_7	日平均负荷过去 7 天滚动均值	2015-01-10	6693.03	8.63	正相关
日平均负荷	load_mean_roll_mean_14	日平均负荷过去 14 天滚动均值	2015-01-10	6235.96	8.15	正相关
日平均负荷	load_mean_lag_7	日平均负荷前 7 天滞后值	2015-01-10	5934.02	7.76	正相关
日平均负荷	temp_avg	平均温度	2015-01-10	15.40	6.68	正相关
日平均负荷	temp_min	最低温度	2015-01-10	12.00	6.67	正相关
日平均负荷	load_mean_lag_14	日平均负荷前 14 天滞后值	2015-01-10	6912.88	6.65	正相关
日平均负荷	doy_cos	年内日周期余弦	2015-01-10	0.99	6.51	负相关
日平均负荷	cdd	制冷度日	2015-01-10	0.00	6.40	正相关
日平均负荷	temp_max	最高温度	2015-01-10	19.90	6.23	正相关
日平均负荷	month_cos	月份周期余弦	2015-01-10	0.87	5.33	负相关
日平均负荷	month_sin	月份周期正弦	2015-01-10	0.50	5.11	负相关
日平均负荷	hdd	供热度日	2015-01-10	2.60	3.90	负相关
日平均负荷	doy_sin	年内日周期正弦	2015-01-10	0.17	3.58	负相关
日平均负荷	is_weekend	是否周末	2015-01-10	1.00	2.12	负相关

目标变量	特征名	中文含义	示例日期	示例值	参考权重 (%)	相关方向
日平均负荷	dow_sin	星期周期正弦	2015-01-10	-0.97	1.62	正相关
日平均负荷	dow_cos	星期周期余弦	2015-01-10	-0.22	1.41	负相关
日平均负荷	humidity	相对湿度	2015-01-10	54.00	1.23	正相关
日平均负荷	load_mean_roll_std_7	日平均负荷过去 7 天滚动标准差	2015-01-10	501.75	1.00	正相关
日平均负荷	load_mean_roll_std_14	日平均负荷过去 14 天滚动标准差	2015-01-10	1033.26	0.86	负相关
日平均负荷	rainfall	降雨量	2015-01-10	0.00	0.76	正相关
日平均负荷	temp_range	日温差	2015-01-10	7.90	0.18	负相关
日最低负荷	load_min_lag_1	日最低负荷前 1 天滞后值	2015-01-10	4841.27	9.47	正相关
日最低负荷	load_min_roll_mean_7	日最低负荷过去 7 天滚动均值	2015-01-10	4461.60	8.79	正相关
日最低负荷	load_min_roll_mean_14	日最低负荷过去 14 天滚动均值	2015-01-10	4210.36	8.41	正相关
日最低负荷	load_min_lag_7	日最低负荷前 7 天滞后值	2015-01-10	3632.87	7.75	正相关
日最低负荷	temp_min	最低温度	2015-01-10	12.00	7.07	正相关
日最低负荷	temp_avg	平均温度	2015-01-10	15.40	7.07	正相关
日最低负荷	cdd	制冷度日	2015-01-10	0.00	6.92	正相关
日最低负荷	load_min_lag_14	日最低负荷前 14 天滞后值	2015-01-10	4826.89	6.90	正相关

目标变量	特征名	中文含义	示例日期	示例值	参考权重(%)	相关方向
日最低负荷	doy_cos	年内日周期余弦	2015-01-10	0.99	6.86	负相关
日最低负荷	temp_max	最高温度	2015-01-10	19.90	6.59	正相关
日最低负荷	month_cos	月份周期余弦	2015-01-10	0.87	5.61	负相关
日最低负荷	month_sin	月份周期正弦	2015-01-10	0.50	5.35	负相关
日最低负荷	hdd	供热度日	2015-01-10	2.60	4.10	负相关
日最低负荷	doy_sin	年内日周期正弦	2015-01-10	0.17	3.72	负相关
日最低负荷	dow_cos	星期周期余弦	2015-01-10	-0.22	1.68	负相关
日最低负荷	humidity	相对湿度	2015-01-10	54.00	1.34	正相关
日最低负荷	rainfall	降雨量	2015-01-10	0.00	0.88	正相关
日最低负荷	load_min_roll_std_7	日最低负荷过去 7 天滚动标准差	2015-01-10	458.42	0.86	正相关
日最低负荷	dow_sin	星期周期正弦	2015-01-10	-0.97	0.36	正相关
日最低负荷	temp_range	日温差	2015-01-10	7.90	0.20	负相关
日最低负荷	load_min_roll_std_14	日最低负荷过去 14 天滚动标准差	2015-01-10	780.47	0.07	负相关
日最低负荷	is_weekend	是否周末	2015-01-10	1.00	0.02	负相关
日最高负荷	load_max_lag_1	日最高负荷前 1 天滞后值	2015-01-10	9000.56	8.49	正相关

目标变量	特征名	中文含义	示例日期	示例值	参考权重(%)	相关方向
日最高负荷	load_max_roll_mean_7	日最高负荷过去 7 天滚动均值	2015-01-10	8575.27	8.41	正相关
日最高负荷	load_max_lag_7	日最高负荷前 7 天滞后值	2015-01-10	7789.19	7.93	正相关
日最高负荷	load_max_roll_mean_14	日最高负荷过去 14 天滚动均值	2015-01-10	8024.68	7.93	正相关
日最高负荷	load_max_lag_14	日最高负荷前 14 天滞后值	2015-01-10	8899.33	6.80	正相关
日最高负荷	temp_min	最低温度	2015-01-10	12.00	6.47	正相关
日最高负荷	temp_avg	平均温度	2015-01-10	15.40	6.47	正相关
日最高负荷	doy_cos	年内日周期余弦	2015-01-10	0.99	6.33	负相关
日最高负荷	temp_max	最高温度	2015-01-10	19.90	6.03	正相关
日最高负荷	cdd	制冷度日	2015-01-10	0.00	6.01	正相关
日最高负荷	month_cos	月份周期余弦	2015-01-10	0.87	5.20	负相关
日最高负荷	month_sin	月份周期正弦	2015-01-10	0.50	4.96	负相关
日最高负荷	hdd	供热量日	2015-01-10	2.60	3.80	负相关
日最高负荷	doy_sin	年内日周期正弦	2015-01-10	0.17	3.47	负相关
日最高负荷	is_weekend	是否周末	2015-01-10	1.00	2.79	负相关
日最高负荷	load_max_roll_std_7	日最高负荷过去 7 天滚动标准差	2015-01-10	566.01	2.75	正相关

目标变量	特征名	中文含义	示例日期	示例值	参考权重(%)	相关方向
日最高负荷	dow_sin	星期周期正弦	2015-01-10	-0.97	2.01	正相关
日最高负荷	dow_cos	星期周期余弦	2015-01-10	-0.22	1.36	负相关
日最高负荷	humidity	相对湿度	2015-01-10	54.00	1.25	正相关
日最高负荷	rainfall	降雨量	2015-01-10	0.00	0.79	正相关
日最高负荷	load_max_roll_std_14	日最高负荷过去 14 天滚动标准差	2015-01-10	1286.21	0.57	正相关
日最高负荷	temp_range	日温差	2015-01-10	7.90	0.20	负相关

代码截图：特征参考权重计算

核心代码 5：特征参考权重计算

```
def compute_feature_weights(daily: pd.DataFrame) -> pd.DataFrame:
    """计算数据处理阶段的特征参考权重，对应 HTML 第 11 节。"""
    rows: list[dict[str, object]] = []
    for target, target_cn in p2.TARGETS.items():
        # 复用主项目里的 make_target_features，保证汇报展示的特征和真实建模特征一致。
        feature_df, features = p2.make_target_features(daily, target)
        feature_df = feature_df[feature_df["date"] <= pd.Timestamp("2015-01-10")]
        # 滞后/滚动特征前几天会产生 NaN，计算相关性前要去掉。
        feature_df = feature_df.dropna(subset=[target, *features])
        scores: list[tuple[str, float, float]] = []
        for feature in features:
            # 这里用“单变量 Pearson 相关系数”做解释性排序，不代表最终模型参数。
            corr = feature_df[[target, feature]].corr().iloc[0, 1]
            if pd.isna(corr) or np.isinf(corr):
                corr = 0.0
            # 权重看强弱，所以取绝对值；正负方向单独保存在“相关方向”里。
            scores.append((feature, float(corr), abs(float(corr))))
        # 把每个目标变量下所有特征的相关性绝对值归一化成百分比。
        total = sum(score for _, _, score in scores) or 1.0
        for feature, corr, score in scores:
            rows.append(
                {
                    "目标变量": target_cn,
                    "特征名": feature,
                    "中文含义": feature_label(feature),
                    "相关方向": "正相关" if corr >= 0 else "负相关",
                    "相关系数": round(corr, 4),
                    "参考权重(%)": round(score / total * 100, 2),
                }
            )
    return pd.DataFrame(rows).sort_values(["目标变量", "参考权重(%)"], ascending=[True, False])
```

```

def compute_feature_weights(daily: pd.DataFrame) -> pd.DataFrame:
    """计算数据处理阶段的特征参考权重，对应 HTML 第 11 节。"""
    rows: list[dict[str, object]] = []
    for target, target_cn in p2.TARGETS.items():
        # 复用主项目里的 make_target_features，保证汇报展示的特征和真实建模特征一致。
        feature_df, features = p2.make_target_features(daily, target)
        feature_df = feature_df[feature_df["date"] <= pd.Timestamp("2015-01-10")]
        # 滞后/滚动特征前几天会产生 NaN，计算相关性前要去掉。
        feature_df = feature_df.dropna(subset=[target, *features])
        scores: list[tuple[str, float, float]] = []
        for feature in features:
            # 这里用“单变量 Pearson 相关系数”做解释性排序，不代表最终模型参数。
            corr = feature_df[[target, feature]].corr().iloc[0, 1]
            if pd.isna(corr) or np.isinf(corr):
                corr = 0.0
            # 权重看强弱，所以取绝对值；正负方向单独保存在“相关方向”里。
            scores.append((feature, float(corr), abs(float(corr))))
        # 把每个目标变量下所有特征的相关性绝对值归一化成百分比。
        total = sum(score for _, _, score in scores) or 1.0
        for feature, corr, score in scores:
            rows.append(
                {
                    "目标变量": target_cn,
                    "特征名": feature,
                    "中文含义": feature_label(feature),
                    "相关方向": "正相关" if corr >= 0 else "负相关",
                    "相关系数": round(corr, 4),
                    "参考权重(%)": round(score / total * 100, 2),
                }
            )
    return pd.DataFrame(rows).sort_values(["目标变量", "参考权重(%)"], ascending=[True, False])

def build_feature_dictionary(daily: pd.DataFrame, weights: pd.DataFrame) -> pd.DataFrame:
    """生成“每个特征的示例值”表，方便汇报时解释特征到底是什么。"""
    rows: list[dict[str, object]] = []
    for target, target_cn in p2.TARGETS.items():
        feature_df, features = p2.make_target_features(daily, target)
        feature_df = feature_df[feature_df["date"] <= pd.Timestamp("2015-01-10")]
        # 取历史期最后一条完整样本作为演示样本，日期接近预测起点，最容易讲清楚。
        sample = feature_df.dropna(subset=features).tail(1).iloc[0]
        weight_part = weights[weights["目标变量"] == target_cn].set_index("特征名")

```

```
for feature in features:
    rows.append(
        {
            "目标变量": target_cn,
            "特征名": feature,
            "中文含义": feature_label(feature),
            "示例日期": sample["date"].strftime("%Y-%m-%d"),
            "示例值": round(float(sample[feature]), 4),
            "参考权重(%)": float(weight_part.loc[feature, "参考权重(%)"]),
            "相关方向": str(weight_part.loc[feature, "相关方向"]),
        }
    )
return pd.DataFrame(rows).sort_values(["目标变量", "参考权重(%)"], ascending=[True, False])
```

12. 第一次汇报结论

经过数据处理，原始 Excel 已转换为可建模的日粒度特征表：目标变量是日最高、日最低、日平均负荷；输入变量包含天气、日历周期、天气衍生变量、历史滞后和滚动统计。新版还补充了缺失值判断、分布检查、极端值识别、物理质量校验、极端事件保留策略，以及每个特征的示例值和参考权重。下一次汇报可以在这个数据集基础上进入算法模型比较、训练过程和最终模型选择。